

Databricks 新 RAG Stack

基于 LinkedIn 帖子与 Databricks 官方文档的核验版实施指南

结论：RAG 没有消失；企业 RAG 正从“切文本 + 向量库”升级为由受治理的数据摄取、文档解析、语义分块、混合检索、重排和应用编排组成的完整系统。

版本：2.0（官方核验版）

日期：2026 年 9 月 28 日

适用：Databricks AI Functions、AI Search、Lakeflow Connect、Databricks Apps

1. 帖子内容的准确总结

Thomas Hass 的帖子反驳“RAG architecture are dead”的观点，核心论点是：即使模型上下文窗口越来越大，也不能把所有企业数据不加选择地塞入上下文；检索、权限、更新、成本和可解释性仍然需要架构。

帖子给出的新链路是：

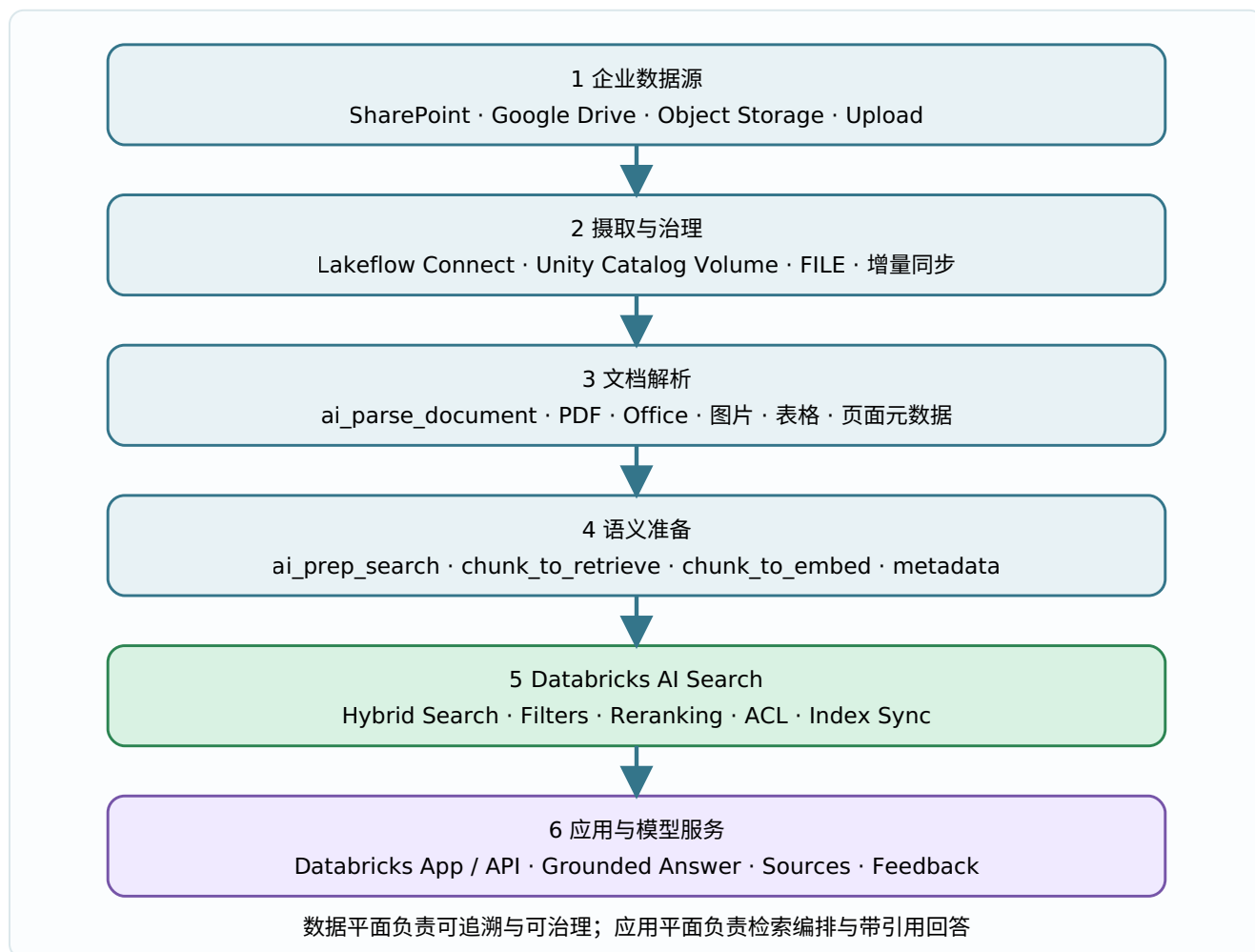
- 1. Ingestion：使用 Lakeflow Connect 从 SharePoint、Google Drive 等来源增量摄取；用 FILE 类型把文件存入 Volume，并在表中保留受治理的指针。
- 2. Parsing：使用 ai_parse_document，以语义合理且带元数据的方式解析文档。
- 3. Chunking：使用新的 ai_prep_search 生成语义合理的 chunk，并增加样例问题等元数据来改善检索。
- 4. Indexing：计算 embedding 并写入 AI Search。
- 5. Retrieval：结合 hybrid search、原生 reranking 和 filters 优化召回。
- 6. Serving：把编排部署为 Databricks App，以获得灵活的 agent orchestration、快速迭代、版本管理和 CI/CD。

帖子与官方文档的对应关系：帖子给出产品组合和实践视角；官方文档确认 AI Functions、AI Search 和 Apps 的接口、限制、权限及运行条件。二者结合后，才形成可落地的生产方案。

2. 官方能力核验

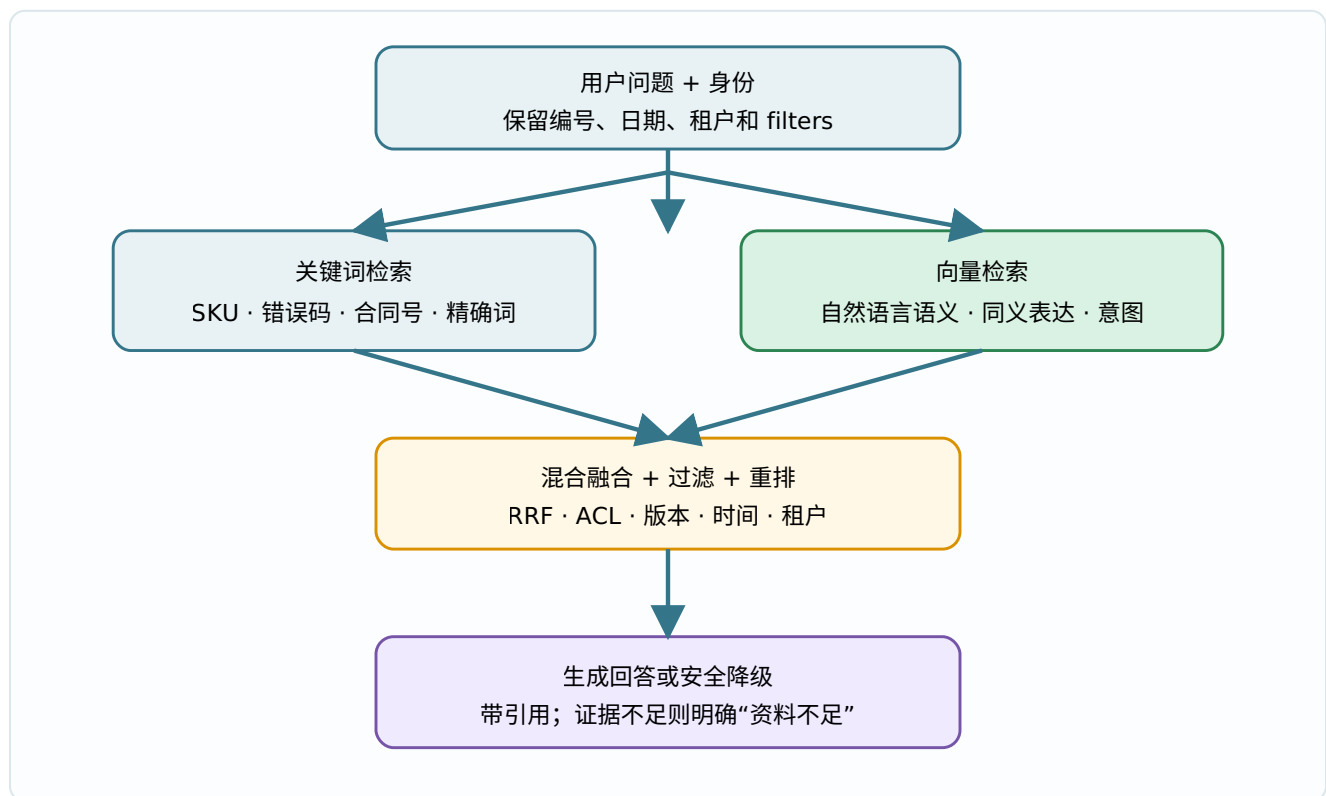
帖子步骤	Databricks 能力	官方核验结果
摄取	Lakeflow Connect、Unity Catalog Volume、FILE	用于把源数据纳入 Lakehouse；增量和权限传播要按具体连接器设计。
解析	ai_parse_document	解析文本、表格、图形描述和版面；支持 PDF、Office、图片等文档类型。
分块	ai_prep_search	Beta；将解析结果或纯文本/Markdown 变成适合 AI Search/RAG 的语义 chunk，并可抽取结构化元数据。
索引	Databricks AI Search	原 Databricks Vector Search；可从 Delta 等表建索引并同步源表更新。
检索	Hybrid、filter、reranking、ACL	官方支持；混合检索用向量相似度与关键词检索结合，RRF 融合排序。
服务	Databricks Apps	可添加 AI Search index resource；应用服务主体需要受治理的 catalog/schema/index 权限。

3. 推荐生产架构



建议将原始文件、解析结果、搜索准备结果和索引源表分层保存。每层保留 document_id、source_uri、源修改时间、内容 hash、版本和 ACL 元数据，避免只能看到“当前答案”而无法复核答案来源。

4. 数据流与检索决策



5. 如何做：端到端实施步骤

5.1 建立原始文件层

1. 在 Unity Catalog 中创建专用 catalog/schema/volume。
2. 配置 Lakeflow Connect 或受控上传；源系统 ID、文件 URI、修改时间和删除状态必须可追踪。
3. 把文件内容和治理元数据分离：文件存储在 Volume，表中保留受治理的文件引用、hash、来源、版本和 ACL。
4. 先定义增量同步、删除传播和权限撤回 SLA，再开始索引。

5.2 解析文档

```
WITH parsed_documents AS (  
  SELECT  
    path,  
    ai_parse_document(content, map('version', '2.0')) AS parsed  
  FROM READ_FILES('/Volumes/my_catalog/my_schema/my_documents/',  
    format => 'binaryFile')  
)  
SELECT path, parsed  
FROM parsed_documents;
```

解析层的输出是结构化 VARIANT，可包含页面、元素、文本、表格和图形描述。需要页码和图像引用时，应按官方选项把页面/元素图像写入 Unity Catalog Volume。REST 调用读取的是 Unity Catalog Volume 路径，而不是任意本地二进制。

5.3 语义分块和元数据

```
WITH parsed_documents AS (  
  SELECT ai_parse_document(content) AS parsed  
  FROM READ_FILES('/Volumes/mydata/documents/', format => 'binaryFile')  
)  
SELECT ai_prep_search(  
  parsed,  
  map('schema', ['company', 'document_type', 'fiscal_year'])  
) AS result  
FROM parsed_documents;
```

ai_prep_search 输出中最重要的字段包括：chunk_id、chunk_position、chunk_to_retrieve、chunk_to_embed、metadata 和页面引用。chunk_to_embed 会把文档标题、页眉、章节、页码、文档上下文和表格摘要等信息与正文组合，以帮助语义检索。

5.4 展平为 AI Search 源表

```
WITH parsed_documents AS (  
  SELECT path, ai_parse_document(content) AS parsed  
  FROM READ_FILES('/Volumes/mydata/documents/', format => 'binaryFile')  
) , prepped_documents AS (  
  SELECT path, ai_prep_search(parsed) AS result  
  FROM parsed_documents  
)  
SELECT  
  chunk.value:chunk_id::STRING AS chunk_id,  
  chunk.value:chunk_position::INT AS chunk_position,  
  chunk.value:chunk_to_retrieve::STRING AS chunk_to_retrieve,  
  chunk.value:chunk_to_embed::STRING AS chunk_to_embed,  
  chunk.value:metadata AS metadata,  
  prepped_documents.path AS source_uri  
FROM prepped_documents,  
  LATERAL variant_explode(prepped_documents.result:document.contents) AS chunk;
```

官方文档明确指出，这个结果可以写入 Delta 表，并以 chunk_to_embed 作为 embedding 输入列、以 chunk_id 作为主键来建立 AI Search index。

5. AI Search：为什么不能只做向量检索

AI Search 支持向量检索、关键词检索、混合检索、过滤、重排和 ACL。混合检索尤其适合包含 SKU、错误码、合同号、产品型号等精确字符串的企业知识库：关键词检索负责命中特定词，向量检索负责捕捉自然语言语义，RRF 负责融合两套结果。

能力	用途	推荐实践
----	----	------

Hybrid	同时处理精确词和语义表达	产品编号、错误码、政策问答默认评估 hybrid
Filter	按部门、租户、版本、日期、语言过滤	把过滤条件放在服务端请求，不信任客户端字段
Reranking	对候选结果做更精细的相关性排序	先扩大候选集，再重排；用黄金问题验证收益
ACL	控制谁能访问搜索 endpoint/index	结合 Unity Catalog 权限和文档级 ACL
Sync	让 index 跟随源表更新	监控索引落后时间和删除传播延迟

6. Databricks App 服务层

在 Databricks App 中添加 AI Search index resource 后，平台可以把 index 的三层名称注入环境变量。应用服务主体至少需要对父 catalog、schema 和 index 具备相应权限；不要把 index 名称、租户 ID 或权限范围完全交给浏览器端控制。

```
env:
  - name: VECTOR_SEARCH_INDEX
    valueFrom: vector-search-index

# Python 示意
import os
from databricks.sdk import WorkspaceClient

index_name = os.environ["VECTOR_SEARCH_INDEX"]
workspace = WorkspaceClient()
results = workspace.vector_search_indexes.query_index(
    index_name=index_name,
    query_text="用户的问题",
    num_results=10,
)
```

生成回答时，应用应把检索结果的正文和引用元数据一起交给模型，并在 UI 中展示文件、页码、版本、更新时间和来源链接。检索不到足够证据时，应返回“资料不足”，而不是强行生成结论。

7. 运行条件与限制

- AI Functions 不能运行在 Classic SQL warehouse；官方总览要求 Databricks Runtime 15.4 LTS 以上，18.2 以上可获得更完整的新能力。
- ai_prep_search 当前是 Beta；要求 DBR 18.2+，Serverless 还要求环境版本 3+，并可在 notebook、SQL editor、workflow、job 或 Lakeflow pipeline 中使用。
- ai_parse_document 的 REST API 只接受 Unity Catalog Volume 路径，官方 API 参考列出单文档 100 页、100 MB 的 REST 限制；SQL/运行时路径的限制要按目标版本文档核验。
- 解析和搜索准备的输出 schema 可能演进；生产表应显式记录 schema version，并避免未经验证就依赖未文档化字段。
- AI Functions 相关模型推理可能产生独立于提交计算的费用；应分别监控 SQL/计算成本与 AI Function 成本。

8. 业务场景

场景	架构重点	验收问题
员工政策助手	生效日期、地区、员工类型过滤	能否引用当前政策并排除旧版本？
客服/运维助手	hybrid + 错误码/型号过滤 + 重排	是否召回正确版本的处置步骤？
合同检索	条款级 chunk、签署状态、强引用	无权限用户是否完全看不到条款？
销售资料助手	产品、市场、日期、机密级别元数据	是否会把内部竞品信息返回给外部用户？
研究报告问答	表格/图形描述、页码、时间范围	结论是否能回到原始图表和页码？

9. 安全与生产清单

- ☐ 源系统、Volume、Delta 表、AI Search index 和 App service principal 均使用最小权限。
- ☐ 检索过滤在服务端执行；ACL 变化、删除和撤回会传播到 index。
- ☐ 每个 chunk 保存 document ID、版本、来源 URI、页码、ACL 和更新时间。
- ☐ 解析失败、索引落后、模型超时和配额错误都有可观测性和告警。
- ☐ 用黄金问题评估 Recall@k、引用正确率、faithfulness、最终答案正确率、延迟和成本。
- ☐ 文档内容被视为不可信输入；提示词注入不能改变系统权限或触发未授权工具。
- ☐ App 的回答显示来源，证据不足时明确降级，不把模型猜测伪装成事实。
- ☐ CI/CD 管理 App、索引配置和 Lakeflow 任务；不依赖个人 notebook 状态。

10. 官方参考

LinkedIn 原帖（Thomas Hass）：https://www.linkedin.com/posts/thomas-hass-53b39b19a_

Databricks AI Functions：<https://docs.databricks.com/aws/en/large-language-models/ai-functions>

ai_prep_search：https://docs.databricks.com/gcp/en/sql/language-manual/functions/ai_prep_search

Document Parsing：<https://docs.databricks.com/aws/en/agents/agent-bricks/document-parsing>

Databricks AI Search：<https://docs.databricks.com/gcp/en/ai-search/ai-search>

AI Search index in Databricks Apps：<https://docs.databricks.com/gcp/en/dev-tools/databricks-apps/vector-search>

本文是基于帖子公开正文和官方文档的中文解释与实施模板；Beta、区域可用性、运行时和 API 限制应在目标工作区再次核验。